Zephyr Project

IT-Schulungen.com
Technology & Transformation

Was ist Zephyr?

Einordnung, Architektur und native_sim

1.03

Zephyr vom Systemmodell bis zur Simulation

Einordnung, Architektur und Einsatzgrenzen auf einen Blick

01

Modelle

Bare Metal, Scheduler, RTOS und Linux

02

Echtzeit

Reaktionszeit, Deadline und Determinismus

03

Kernel

Threads, Timer und Synchronisation

04

Ökosystem

Treiber, Netzwerk, Shell und Sicherheit

05

Simulation

Nutzen und Grenzen von native_sim

Ziel: erklären können, wann ein RTOS sinnvoll ist und welche Rolle Zephyr übernimmt.

Vier Systemmodelle im Vergleich

Bare Metal	Eine Hauptschleife, direkte Hardwaresteuerung und maximale Kontrolle
Scheduler	Mehrere Aufgaben werden geplant, aber noch ohne vollständiges Betriebssystem
RTOS	Threads, Prioritäten und definierte Reaktionszeiten für eingebettete Systeme
General-Purpose-OS	Durchsatz, Komfort und Isolation; nicht primär auf harte Deadlines optimiert
Leitfrage	Welche Reaktion muss bis wann zuverlässig erfolgt sein?

Ein RTOS wird wertvoll, wenn mehrere Aufgaben zeitlich koordiniert und vorhersagbar reagieren müssen.

Was bedeutet Echtzeit?

BEGRIFFE

```
Zeitliche Korrektheit = Ergebnis + rechtzeitige Reaktion
```

Reaktionszeit	Zeit vom Ereignis bis zur sichtbaren Systemreaktion
Deadline	Spätester Zeitpunkt, zu dem ein Ergebnis vorliegen muss
Determinismus	Verhalten bleibt innerhalb bekannter zeitlicher Grenzen
Jitter	Schwankung zwischen erwarteter und tatsächlicher Ausführung

Echtzeit bedeutet nicht maximal schnell, sondern innerhalb definierter Grenzen vorhersehbar.

Zephyr im Gesamtsystem

SCHICHTEN

ANWENDUNG

Subsysteme & APIs

Zephyr-Kernel

Treiber

Devicetree

Hardware / native_sim

01 Anwendung

Domänenspezifisches Verhalten nutzt stabile Zephyr-APIs.

02 Kernel & Subsysteme

Scheduling, Zeitdienste, Synchronisation und Systemdienste.

03 Treiber & Hardware

Treiber greifen zu; Devicetree beschreibt die verfügbare Hardware.

Klare Schichten trennen portable Anwendungslogik von hardwarenaher Implementierung.

Aufgaben des Zephyr-Kernels

KERNEL-KERN

Threads werden nach Priorität und Zustand geplant.

Threads & Scheduling	Prioritäten, Zustände und Wechsel zwischen ausführbaren Threads
Timer & Zeitdienste	Verzögerungen, Timeouts und zeitgesteuerte Aktionen
Synchronisation	Mutexe, Semaphore, Queues und Events koordinieren Nebenläufigkeit
Speicher	Stacks, Heaps und optionale Schutzmechanismen verwalten Ressourcen
Fehlerbehandlung	Assertions, Watchdogs und Diagnosepfade machen Fehler sichtbar

Der Kernel koordiniert Ausführung und Ressourcen; die Anwendung beschreibt das Verhalten.

Zephyr bringt mehr als einen Kernel mit

TREIBER & I/O

Einheitliche APIs

GPIO, UART, I²C, SPI

Sensoren und Aktoren

Boardunabhängiger Zugriff

KOMMUNIKATION

Protokolle & Speicher

Bluetooth, TCP/IP, CAN

USB und Dateisysteme

Modular auswählbar

DIAGNOSE & SCHUTZ

Betrieb sichtbar machen

`Logging und Shell`

`Tests und Tracing`

`Sicherheitsfunktionen`

Subsysteme werden über Kconfig ausgewählt und passend zur Zielhardware gebaut.

Kconfig, Devicetree und Build-System

ZUSAMMENSPIEL

```
Kconfig        → Funktionen
Devicetree     → Hardware
CMake + west   → Build
Generated      → Schnittstelle
```

01 Kconfig

Legt fest, welche Softwarefunktionen aktiv sind und prüft Abhängigkeiten.

02 Devicetree & Build

Beschreiben Hardware und übersetzen Auswahl, Quellen und Toolchain in Firmware.

Konfiguration, Hardwarebeschreibung und Build-System erzeugen gemeinsam die konkrete Firmware.

Breite Plattformbasis, klarer Einsatzfokus

01	**Architekturen**	mehrere MCU- und CPU-Familien mit gemeinsamen Kernel-APIs
02	**SoCs & Boards**	Boardbeschreibungen, Treiber und Samples verbinden Software und Hardware
03	**Typische Produkte**	vernetzte Sensoren, Steuerungen, Wearables und kompakte Gateways

Zephyr ist stark, wenn knappe Hardware, Nebenläufigkeit und Konnektivität zusammentreffen.

native_sim: schnell lernen, früh testen

HOST-AUSFÜHRUNG

```
$ west build -b native_sim app
$ ./build/zephyr/zephyr.exe
Zephyr läuft als Linux-Prozess
```

01 **Stark für**

Anwendungslogik, Threads, IPC, Tests und CI

02 **Nicht ausreichend für**

MCU-Timing, elektrisches Verhalten und konkrete Peripherie

native_sim verkürzt die Rückkopplung, ersetzt aber nicht die Prüfung auf Zielhardware.

Wann ist Zephyr die richtige Wahl?

ENTSCHEIDUNG

Gute Passung

Mehrere parallele Aufgaben
Vorhersagbare Reaktionen
Modulare Systemdienste
Knappe Ressourcen

ARCHITEKTUR

Was Zephyr beiträgt

Kernel und Treiber
Protokolle und Dienste
Konfiguration und Tests
Portierbare APIs

NÄCHSTER SCHRITT

Mini-Übung 1.04

```
System in Schichten zerlegen
Zeitkritisches markieren
Simulationsgrenzen benennen
Ergebnis kurz vorstellen
```

Als Nächstes übertragen wir das Modell auf ein konkretes eingebettetes System.